

Global E-Commerce Order Management Platform (POC)

Version 1 – Project Details and local setup steps

Architecture Overview

Angular Frontend → ASP.NET Core 10 Web API → Application Layer → Infrastructure (Dapper)
→ SQLite Database

Version 1 Features

- Product Catalog
- Shopping Cart
- Checkout
- Mock Payment
- Order Creation
- Order Confirmation
- Angular Material UI
- SQLite Database
- API Versioning
- FluentValidation
- Clean Architecture

Technology Stack

Backend

- .NET 10
- ASP.NET Core Web API
- Dapper
- SQLite
- FluentValidation

Frontend

- Angular
- TypeScript
- Angular Material
- Signals
- Reactive Forms

Prerequisites

- .NET 10 SDK
- Node.js
- Angular CLI

Steps for Local setup –

Clone Repository

git clone <https://github.com/rsrohithsingh423/GlobalECommerce.git>

cd GlobalECommerce

Backend Setup

1. cd backend
2. dotnet restore
3. dotnet build
4. dotnet run

Swagger: <https://localhost:<port>/swagger>

SQLite Database

The application uses SQLite. If no database exists it will be created automatically on first run.

Frontend Setup

5. cd frontend/global-ecommerce-ui
6. npm install
7. ng serve

Application URL: <http://localhost:4200>

Update the API URL if required in below file

src/environments/environment.ts

Running the Application

8. Start the backend
9. Start the Angular application
10. Open <http://localhost:4200>
11. Browse products → Add to Cart → Checkout → Mock Payment → Order Confirmation

Project Structure

backend

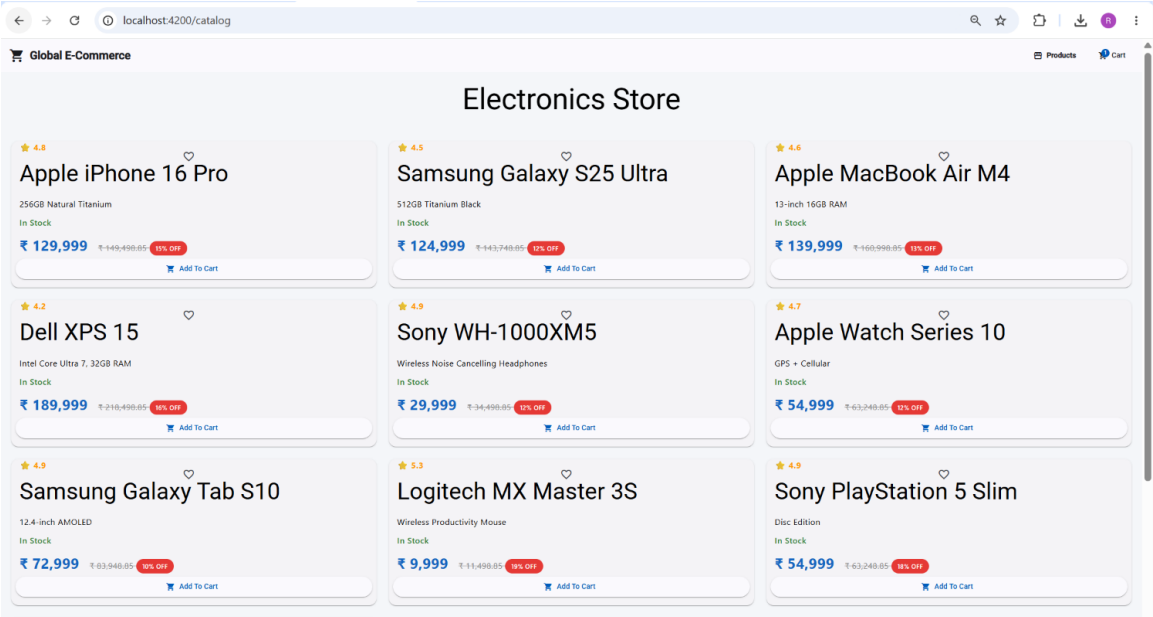
- ├── GlobalECommerce.Api
- ├── GlobalECommerce.Application
- ├── GlobalECommerce.Domain
- └── GlobalECommerce.Infrastructure

frontend

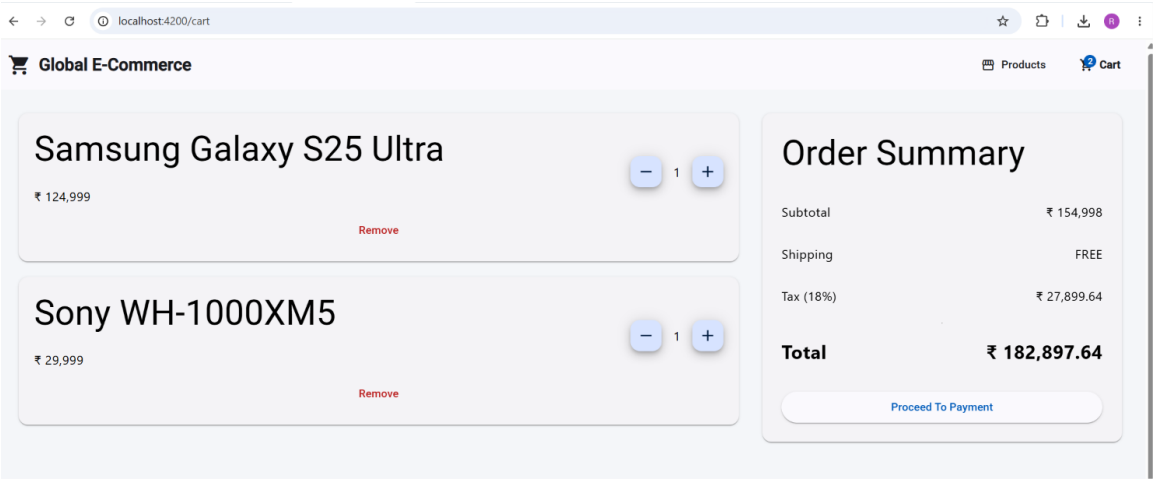
- └── global-ecommerce-ui

Screen Shots of User Journey

Product List



Checkout:



Payment:

← → ↻ localhost:4200/payment ☆ 📄 📄 📄

🛒 Global E-Commerce 📄 Products 🛒 Cart

Secure Payment

Card Holder Name*
Rohit Singh

Card Number*
1234567890123456

MM*
12

YYYY*
2026

CVV*
...

Order Summary

Items : 2

₹ 182,897.64

Pay Now

Order Successful:

🛒 Global E-Commerce 📄 Products 🛒 Cart



Order Confirmed!

Thank you for your purchase.

Order # **ORD-20260706175903**

**Payment**
Successful

**Shipping**
3-5 Business Days

**Status**
Preparing Order

Architecture Philosophy for this POC

The original requirement targets:

- Millions of orders/day
- Event-driven Microservices
- Cloud-native deployment
- High availability

For this Proof of Concept, I intentionally implemented a **Modular Monolith**.

This keeps the project:

- Easy to clone
- Easy to run locally
- Easy to review
- Infrastructure independent

while maintaining module boundaries that can later evolve into independent microservices.

Reasons:

- Demonstrates functionality without requiring cloud infrastructure.
- Simplifies local development and evaluation.
- Avoids overhead of managing multiple services, databases, message brokers, and deployment pipelines.
- Enables reviewers to clone and run the project with minimal setup.

The application is still organized into independent modules (Catalog, Orders, Payment), allowing a straightforward migration to microservices in the future.